Module Descriptor

Programme	Master of Science (MSc) in Computer Science
Module Code	M12
Module Title	Data Structures and Algorithms
MQF/EQF Level	Level 7
ECTS Credits	12
Total Learning Hours	300 hours
Contact Hours	60 hours (20 sessions × 3 hours)
Self-Study Hours	212 hours
Assessment Hours	28 hours
Mode of Delivery	Face-to-Face
Teaching Methods	Lectures, Lab Sessions, Tutorials
Assessment	Written Exam (60%) | Assignments (40%) — algorithmic problem sets and implementation tasks
Status	Compulsory for CS track; Elective for other MSc tracks

Module Description

Data Structures and Algorithms is the foundational engineering module of the MSc in Computer Science, providing students with the rigorous design, analysis, and implementation skills required for professional software engineering and research. The module examines how choice of data structure and algorithm profoundly impacts the performance, scalability, and correctness of software systems.

The module covers a comprehensive range of abstract data types and their concrete implementations: arrays, dynamic arrays, linked lists, stacks, queues, trees (binary search trees, AVL trees, red-black trees, B-trees), heaps, hash tables, graphs, and disjoint sets. For each structure, students analyse time and space complexity in terms of Big-O, Big-Omega, and Big-Theta notation, and consider real-world performance trade-offs including cache behaviour and memory allocation.

The algorithms curriculum spans comparison-based sorting (insertion, merge, quicksort, heapsort), linear-time sorting (counting, radix, bucket sort), searching and selection, dynamic programming (Fibonacci, matrix chain multiplication, longest common subsequence, knapsack), greedy algorithms (activity selection, Huffman coding), and graph algorithms (BFS, DFS, topological sort, Dijkstra, Bellman-Ford, Floyd-Warshall, Prim, Kruskal). Amortised analysis is used for dynamic arrays and splay trees. The module concludes with NP-completeness and approximation strategies, bridging into the theoretical content of M11.

Implementation is emphasised throughout using Python and C++. Students complete four programming assignments, each requiring both correct implementation and empirical

performance analysis. The final session integrates the module through a competitive programming workshop and exam preparation.

Learning Outcomes

Competences

- Design and implement fundamental data structures with correct time and space complexity
- Select appropriate data structures and algorithms for a given problem, justifying the choice analytically
- Analyse algorithm correctness using loop invariants and recurrence relations
- Apply dynamic programming and greedy paradigms to optimisation problems
- Implement and evaluate graph algorithms for shortest path, spanning tree, and traversal problems
- Measure and compare empirical performance of algorithms against theoretical predictions

Knowledge and Understanding

- Define arrays, linked lists, stacks, queues, trees, heaps, hash tables, and graphs as abstract data types
- Recall Big-O, Big-Omega, Big-Theta notation and apply them to classify algorithm complexity
- Describe the Master Theorem and apply it to divide-and-conquer recurrences
- Identify the properties of comparison-based sorting lower bounds (Omega(n log n))
- Understand the P vs NP relationship and its implications for algorithm design
- Recall amortised analysis methods: aggregate, accounting, and potential methods

Skills

- Implement binary search trees, AVL trees, and hash tables with chaining and open addressing
- Trace quicksort, mergesort, and heapsort on sample inputs, accounting for worst/average/best cases
- Write dynamic programming solutions with memoisation and bottom-up tabulation
- Apply BFS and DFS systematically, including cycle detection and topological ordering
- Run Dijkstra's and Bellman-Ford shortest path algorithms on weighted directed graphs
- Profile Python and C++ implementations and interpret performance benchmarks

Assessment

Written Examination — 60%

Closed-book written examination covering: algorithm analysis (recurrences, asymptotic notation), data structure operations and complexity proofs, trace executions of sorting and

graph algorithms, dynamic programming problem formulation, and NP-completeness. 2.5 hours.

Assignments — 40%

- Assignment 1 (Session 5): Implement and benchmark array-based, linked list, and BST-based implementations of a symbol table; compare empirically
- Assignment 2 (Session 10): Sorting algorithm implementation and empirical comparison; include heapsort, quicksort (3-way partition), and mergesort with timing on 10^4 to 10^7 inputs
- Assignment 3 (Session 15): Dynamic programming problems — implement LCS, matrix chain multiplication, and 0/1 knapsack with memoisation
- Assignment 4 (Session 19): Graph algorithm implementations — BFS, DFS, Dijkstra, Kruskal, and Prim on a real-world dataset (e.g., road network, social graph)

Core Reading List

- Cormen, T.H., Leiserson, C.E., Rivest, R.L. & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.
- Goodrich, M.T., Tamassia, R. & Goldwasser, M.H. (2013). Data Structures and Algorithms in Python. Wiley.
- Sedgewick, R. & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley.
- Skiena, S.S. (2020). The Algorithm Design Manual (3rd ed.). Springer.